



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOPs and Flutter Widgets
FireBase

Dr. Annu Sharma

Department of Computer Applications

FireBase Introduction

- Firebase is a platform developed by Google to help you build and manage mobile and web applications.
- It offers a variety of backend services that allow you to focus on your app's frontend without having to worry much about managing servers or infrastructure.
- Firebase is widely used for mobile app development, especially for iOS and Android, and it also supports web apps.

Key Features of Firebase

Firestore provides a suite of tools that can be divided into three main categories:

1. Build
2. Release & Monitor
3. Engage

Key Features of Firebase

1. Build:

- **Firebase Realtime Database:** A NoSQL cloud database that stores and syncs data in real-time across clients. Ideal for apps that need to display live updates or changes.
- **Cloud Firestore:** A flexible, scalable database that also allows real-time syncing. It's more powerful and has better querying capabilities than the Realtime Database.
- **Firebase Authentication:** A feature that simplifies user authentication with email, Google, Facebook, and other providers, making it easier to secure user data.

Key Features of Firebase

1. Build:

- **Firebase Realtime Database:** A NoSQL cloud database that stores and syncs data in real-time across clients. Ideal for apps that need to display live updates or changes.
- **Cloud Firestore:** A flexible, scalable database that also allows real-time syncing. It's more powerful and has better querying capabilities than the Realtime Database.
- **Firebase Authentication:** A feature that simplifies user authentication with email, Google, Facebook, and other providers, making it easier to secure user data.

Key Features of Firebase

1. Build:

- **Firebase Cloud Functions:** A serverless solution to run backend code in response to events triggered by Firebase features and HTTPS requests.
- **Firebase Hosting:** Provides secure, fast, and reliable static and dynamic web hosting.
- **Firebase Cloud Storage:** Lets you store and serve user-generated content, such as photos or videos.

Key Features of Firebase

Release & Monitor:

- **Crashlytics:** A tool for real-time crash reporting and analytics to help you find and fix issues in your app quickly.
- **Performance Monitoring:** Allows you to monitor your app's performance and optimize it for a better user experience.
- **Test Lab:** Provides a way to test your app on various real devices hosted by Google, which helps you identify compatibility issues.

Key Features of Firebase

Engage:

- **Firebase Cloud Messaging (FCM):** A messaging service that lets you send notifications and messages to engage users.
- **In-App Messaging:** Enables you to show in-app messages to guide users or provide information based on their activity.
- **Remote Config:** Allows you to dynamically change your app's behavior or appearance without releasing a new version.
- **Analytics:** Provides insights into user behavior and engagement to help you make data-driven decisions.

Advantages of Using Firebase:

- **Scalability:** Firebase services are managed by Google, so they can handle large user bases without manual scaling.
- **Real-Time Updates:** Both the Realtime Database and Firestore support real-time updates, which is ideal for chat apps, live feeds, and other applications requiring live data.
- **Cross-Platform Support:** Firebase has libraries for iOS, Android, and Web, making it easy to develop cross-platform applications.
- **Serverless Approach:** With Cloud Functions and other backend services, you don't need to manage servers, which reduces infrastructure and DevOps overhead.
- **Free and Flexible Pricing:** Firebase has a free plan that offers a lot of functionality, and a pay-as-you-go pricing model for when you need more resources.

When to Use Firebase

- Firebase is ideal for small to medium-sized applications or MVPs, where the focus is on rapid development and ease of integration.
- It is also popular among startups and small teams because it reduces the need for a dedicated backend developer.
- Firebase can also be scaled for larger apps, though large enterprises with custom backend requirements might still prefer their own infrastructure.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

async and await & Future

Dr. Annu Sharma

Department of Computer Applications

async Introduction

- In Flutter, the `async` keyword is used to declare a function as asynchronous, which means it can run independently of the program's natural flow.
- Asynchronous functions are often used for time-consuming tasks, such as fetching data from a server, reading or writing to a database, or downloading from the internet.

await Introduction

- The **await** keyword is used to work with asynchronous operations, allowing you to wait for a **Future** to complete before proceeding to the next line of code.
- It simplifies handling asynchronous code by making it more readable and synchronous-looking, even though it still runs asynchronously under the hood.

await Introduction

Syntax:

```
Future<void> fetchData() async {  
    var data = await someAsyncFunction();  
    print(data);  
}
```

return type:

Functions using async typically return a Future. If the function doesn't explicitly return a value, the return type is Future<void>.

await keyword:

Pauses the execution of the function until the Future completes. Other code outside the function can continue executing.

Future

Dart is a single-threaded programming language. `Future<T>` object represents the result of an asynchronous operation which produces a result of type `T`.

A Future represents a single value either a data or an error asynchronously

There are 2 ways to handle Futures:

Using the Future API

Using the `async` and `await` operation.

Future

```
Future delayedPrint(int seconds, String msg) {  
  
  final duration = Duration(seconds: seconds);  
  return Future.delayed(duration).then((value) => msg);  
}  
  
main() async {  
  print('PES');  
  await delayedPrint(2, "Is").then((status){  
    print(status); });  
  print('Best'); }
```

Mobile App Development

Flutter Widgets : async and await & future



```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      home: AsyncAwaitExample(),  
    );  
  }  
}
```

```
class AsyncAwaitExample extends  
StatefulWidget {  
  const AsyncAwaitExample({super.key});  
  @override  
  State<AsyncAwaitExample> createState()  
=> _AsyncAwaitExampleState();  
}
```

```
class _AsyncAwaitExampleState extends  
State<AsyncAwaitExample> {  
  String data = "Press the button to fetch  
data";  
  // Simulate an async operation
```

Mobile App Development

Flutter Widgets : Program contd.



```
// Simulate an async operation
Future<String> fetchData() async {
  await Future.delayed(const Duration(seconds: 3)); // simulate delay
  return "Data fetched successfully!";
}
```

```
void getData() async {
  setState(() {
    data = "Fetching data...";
  });
}
```

```
String result = await fetchData(); // Wait for the Future to complete
```

```
setState(() {
  data = result;
});
```

Mobile App Development

Flutter Widgets : Program contd.



```
setState(() {  
  data = result;  
});  
}  
  
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text("Async & Await Example")),  
    body: Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          Text(  
            data
```

Mobile App Development

Flutter Widgets : async & await & futures Program



contd.
Text(

```
    data,  
    textAlign: TextAlign.center,  
    style: const TextStyle(fontSize: 20),  
  ),  
  const SizedBox(height: 20),  
  ElevatedButton(  
    onPressed: getData,  
    child: const Text("Fetch Data"),  
  ),  
],  
,  
,  
,  
);  
}  
}
```


1. What does the `await` keyword do in Dart?

- a) Stops the whole program until the future completes
- b) Waits only within the `async` function without blocking the main thread
- c) Runs multiple tasks simultaneously
- d) Converts Future into Stream

Answer: b) Waits only within the `async` function without blocking the main thread

2. Which of the following functions returns a `Future` object in Flutter?

- a) `print()`
- b) `runApp()`
- c) `http.get()`
- d) `Text()`

Answer: c) `http.get()`

5. What happens if `await` is used in a non-async function?

- a) It will compile normally
- b) It will throw a runtime error
- c) It will show a compile-time error
- d) It will automatically convert to async

Answer: c) It will show a compile-time error

6. Which statement about Dart's main thread is true?

- a) Async tasks block the main thread
- b) Async tasks run on the main thread synchronously
- c) Async tasks run without blocking the main thread
- d) Async tasks require separate threads

Answer: c) Async tasks run without blocking the main thread



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Firestore Configuration json File

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

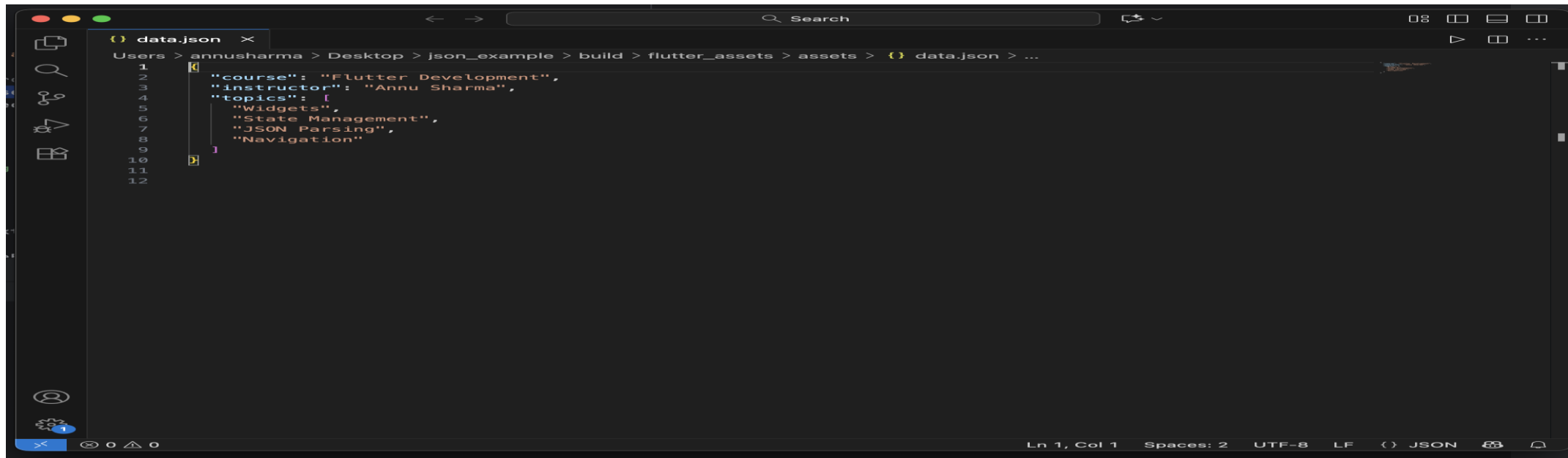
Flutter

Firestore Configuration json File

Create a file named:

assets/data/student.json

Inside the file, put this JSON content:



```
1  {
2    "course": "Flutter Development",
3    "instructor": "Annu Sharma",
4    "topics": [
5      "Widgets",
6      "State Management",
7      "JSON Parsing",
8      "Navigation"
9    ]
10  }
```

Firestore Configuration json File

Step 2: Add JSON file to pubspec.yaml

Open pubspec.yaml and add:

```
flutter: assets: - assets/data/student.json
```

Firestore Configuration json File

Step 3: Read this JSON file in Flutter (Dart)

```
import 'dart:convert';import 'package:flutter/services.dart';  
Future<void> readJson()  
async { final String response =  
await rootBundle.loadString('assets/data/student.json');  
final data = await json.decode(response); print(data["students"]);}
```


Firestore Configuration json File

Call this method in initState():

```
@override void initState()  
{  
  super.initState(); readJson();  
}
```

Mobile App Development

Flutter



Firestore Configuration json File

```
main.dart x
1  import 'dart:convert';
2  import 'package:flutter/material.dart';
3  import 'package:flutter/services.dart' show rootBundle;
4
5  void main() => runApp(const MyApp());
6
7  class MyApp extends StatefulWidget {
8    const MyApp({super.key});
9
10   @override
11   State<MyApp> createState() => _MyAppState();
12 }
13
```

Firestore Configuration json File

Method initState()

```
13    
14  class _MyAppState extends State<MyApp> {}  
15      Map<String, dynamic>? jsonData;  
16  
17      @override  
18  void initState() {  
19      super.initState();  
20      loadJson();  
21  }
```

Firestore Configuration json File

Future Keyword method LoadJson

```
22
23 Future<void> loadJson() async {
24     try {
25         // Load JSON file from assets
26         final String response = await rootBundle.loadString('assets/data.json');
27         final data = json.decode(response);
28         setState(() {
29             jsonData = data;
30         });
31     } catch (e) {
32         print("Error loading JSON: $e");
33     }
34 }
35
```

Firestore Configuration json File

```
35
36  @override
37  Widget build(BuildContext context) {
38    return MaterialApp(
39      debugShowCheckedModeBanner: false,
40      home: Scaffold(
41        appBar: AppBar(title: const Text('Flutter JSON Example')),
42        body: jsonData == null
43          ? const Center(child: CircularProgressIndicator())
44          : Padding(
45            padding: const EdgeInsets.all(16),
46            child: Column(
47              crossAxisAlignment: CrossAxisAlignment.start,
48              children: [
```

Mobile App Development

Flutter



```
47   crossAxisAlignment: CrossAxisAlignment.start,
48   children: [
49     Text('Course: ${jsonData!["course"]}',
50       style: const TextStyle(fontSize: 18)), Text
51     Text('Instructor: ${jsonData!["instructor"]}',
52       style: const TextStyle(fontSize: 18)), Text
53     const SizedBox(height: 10),
54     const Text('Topics:',
55       style: TextStyle(
56         fontSize: 20, fontWeight: FontWeight.bold)), Text TextStyle
57     ...List<Widget>.from(
58       (jsonData!["topics"] as List).map(
59         (topic) => Text('• $topic',
```

Mobile App Development

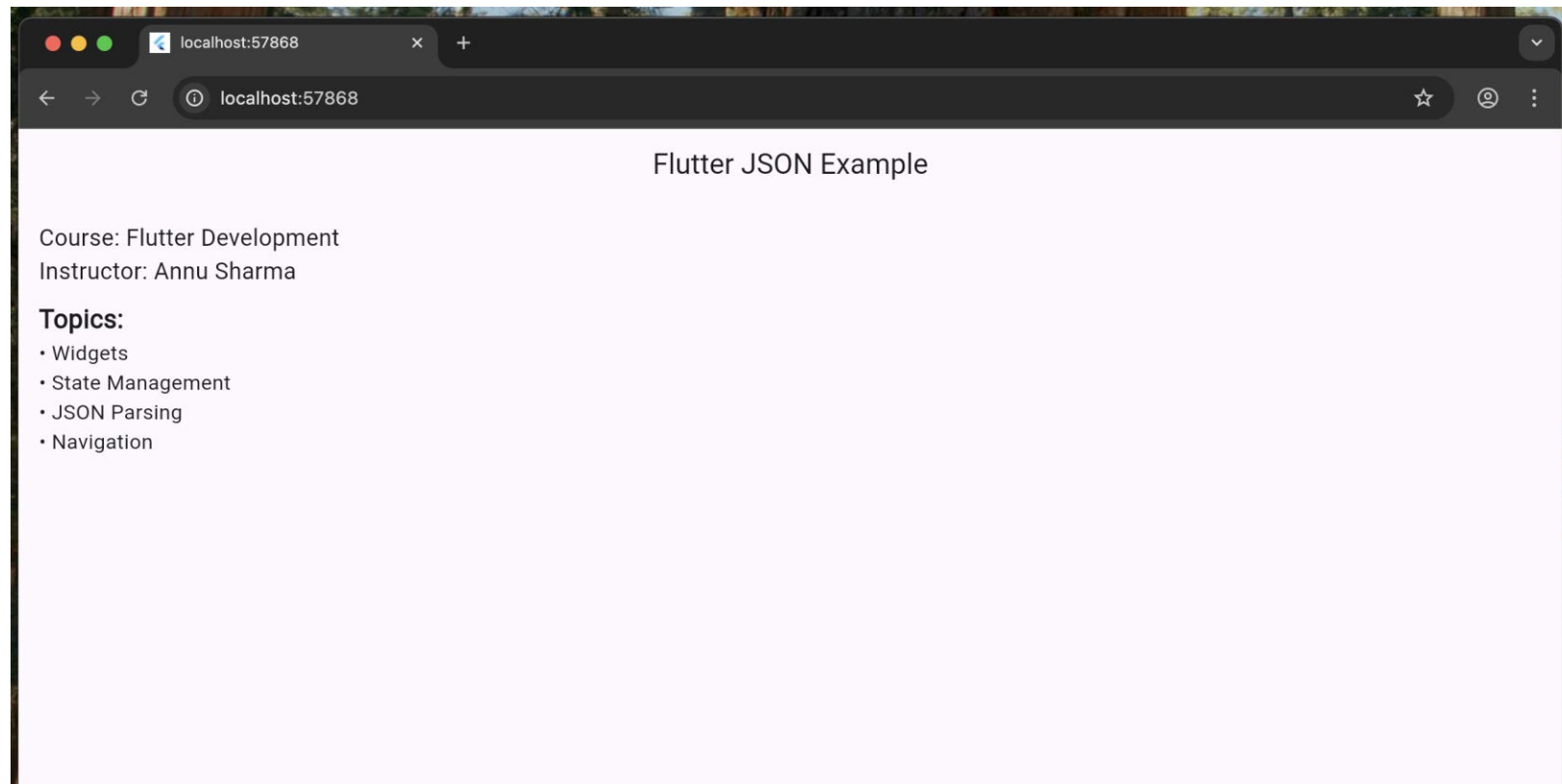
Flutter

```
53     const SizedBox(height: 10),
54     const Text('Topics:',
55       style: TextStyle(
56         fontSize: 20, fontWeight: FontWeight.bold)), Text TextStyle
57     ...List<Widget>.from(
58       (jsonData!["topics"] as List).map(
59         (topic) => Text('• $topic',
60           style: const TextStyle(fontSize: 16)), Text
61       ),
62     ), List.from
63   ],
64 ), Column
65 ), Padding
66 ), Scaffold
67 ); MaterialApp
68 }
69 }
70
```

Mobile App Development

Flutter

Output





THANK YOU

Dr. Annu Sharma

Associate Professor

Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

JSON Data Processing
Getting Started with Firebase Database

Dr. Annu Sharma

Department of Computer Applications

- Firebase databases (Real-Time Database and Firestore) store data in JSON format.
- The *dart:convert* library provides methods like *jsonEncode* and *jsonDecode* to convert JSON data to and from Dart objects.

```
import 'dart:convert';

class User {
    final String name;
    final int age;

    User({required this.name, required this.age});

    factory User.fromJson(Map<String, dynamic> json) => User(name: json['name'], age:
    json['age'], );

    Map<String, dynamic> toJson() => {'name': name, 'age': age};
}
```

```
void main() {
```

```
    String jsonString = '{"name": "Atharv", "age": 20}';
```

```
    Map<String, dynamic> userMap = jsonDecode(jsonString);
```

```
    User user = User.fromJson(userMap);
```

```
    print("Name : ${user.name}");
```

```
    print("Age : ${user.age}");
```

```
}
```

Output

```
Name : Atharv  
Age : 20
```

Create a student JSON file that contains multiple student objects. Each student has following key attributes :

- id: A unique identifier for the student.
- name: The student's full name.
- age: The student's age.
- gender: The student's gender.
- email: The student's email address.
- mobile: The student's mobile number.
- courses: An array that contain details of the course:
 - courseId: A unique identifier for the course.
 - courseName: The name of the course.
 - subject: An array that list the subject the student in undertaking in the enrolled course
 - subject_code:
 - subject_name:
 - credits:

Mobile App Development

JSON File Structure



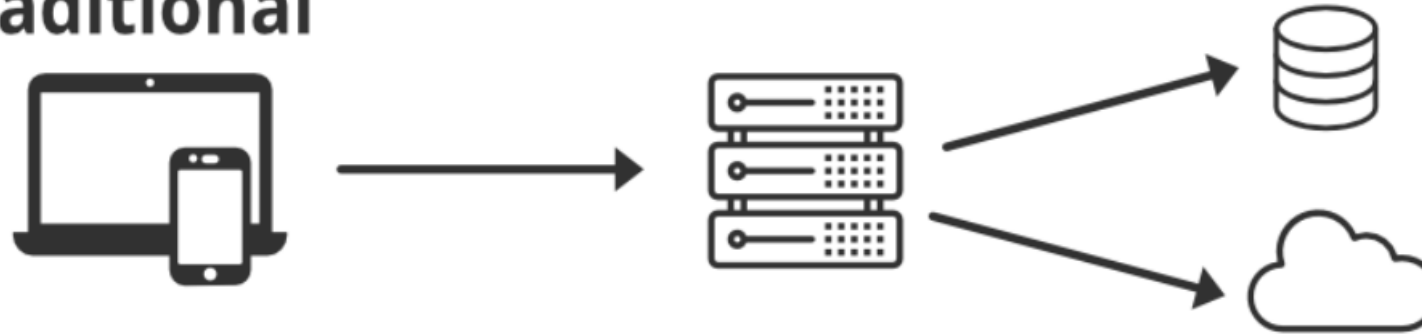
```
students (array)
├── student (object)
│   ├── id (string)
│   ├── name (string)
│   ├── age (number)
│   ├── gender (string)
│   ├── email (string)
│   ├── mobile (string)
│   └── course (object)
│       ├── courseId (string)
│       ├── courseName (string)
│       └── subjects (array)
│           ├── subject (object)
│           │   ├── subject_code (string)
│           │   ├── subject_name (string)
│           │   └── credits (number)
│           └── ...
└── ...
```

- **students (array):** The top-level array that holds multiple student objects.
- **Each student (object):** Contains basic details of a student (id, name, age, gender, email, and mobile).
- Has a single nested **course** object.
- **course (object):** Contains details about the course (courseId and courseName).
- Has a nested array of **subjects**.
- **Each subject (object):** Contains attributes specific to the subject (subject_code, subject_name, and credits).

Mobile App Development

Getting Started with Firebase Database

Traditional



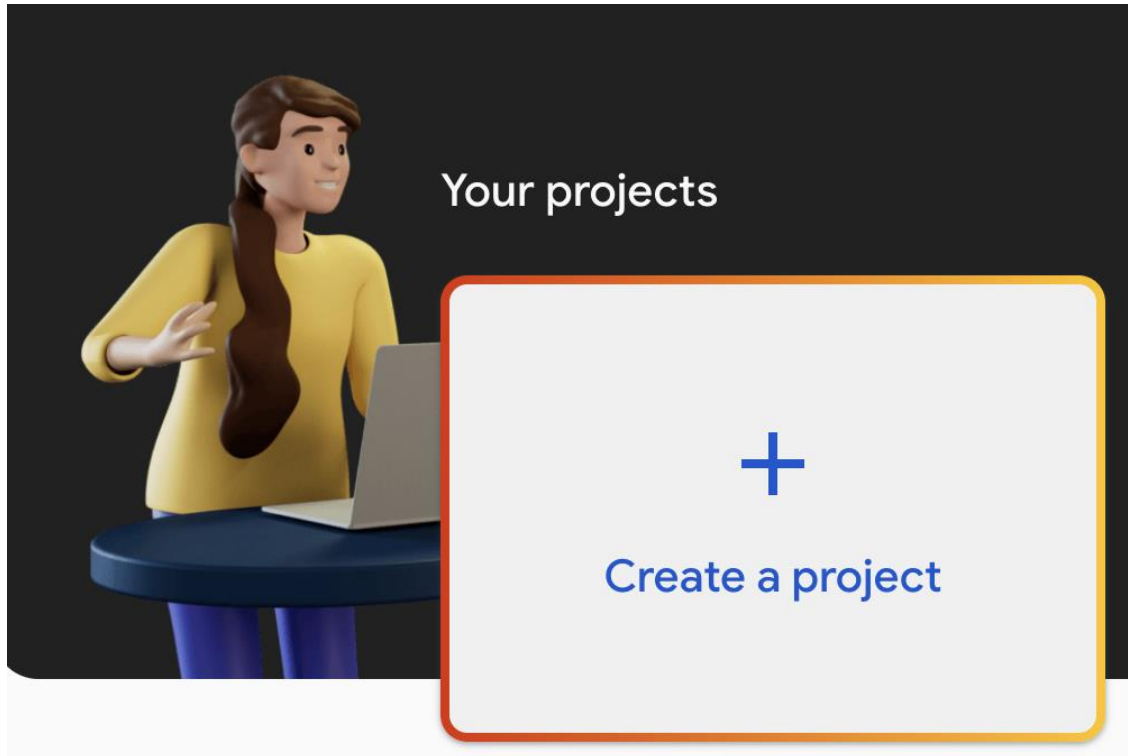
Firebase



Mobile App Development

Getting Started with Firebase Database

Click on Create a project



Enter a Valid project name

× Create a project

Let's start with a name for your project[?]

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue

Mobile App Development

Getting Started with Firebase Database



Step 1: Create a Firebase Project

1. Go to Firebase Console.
2. Click on **Add Project**.
3. Name your project and follow the setup prompts.
4. Once created, you'll be directed to the Firebase project dashboard.
5. **Firestore Database** : Create JSON Database (Manually) -- Select Build(project dashboard) > Firestore Database



THANK YOU

Dr.Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Working with Firebase

Dr. Annu Sharma

Department of Computer Applications

Firestore Introduction

- **Firestore is a platform by Google that provides tools for building, scaling, and managing mobile and web applications.**
- **Offers services for real-time databases, authentication, analytics, and machine learning.**

Firestore's Purpose:

- Designed for developers to quickly build high-quality apps and improve engagement.

Core Products:

- Firestore Realtime Database, Cloud Firestore, Firestore Authentication, and Firestore Analytics.

Firestore Realtime Database:

Structure: Uses a NoSQL JSON tree format.

Ideal For: Real-time, collaborative applications like chat apps, live maps, or shared document editing.

Strengths:

Real-time syncing, fast response times, simplicity.

Cloud Firestore

Structure: Document-oriented, with collections and documents for complex data structures.

Ideal For: Applications with advanced querying needs and offline-first experience.

Strengths: Scalable, strong querying capabilities, supports nested data.

Features of Firebase Realtime Database

Real-Time Data Sync:

Data changes are instantly propagated to all connected clients.

Offline Access:

Data is cached locally on the device and synced when back online.

Scalable Architecture:

Supports both small and large applications with flexible data scaling.

Security with Firebase Authentication & Rules:

Built-in user authentication integrates seamlessly, enabling secure access control.

Integration with Firebase Analytics:

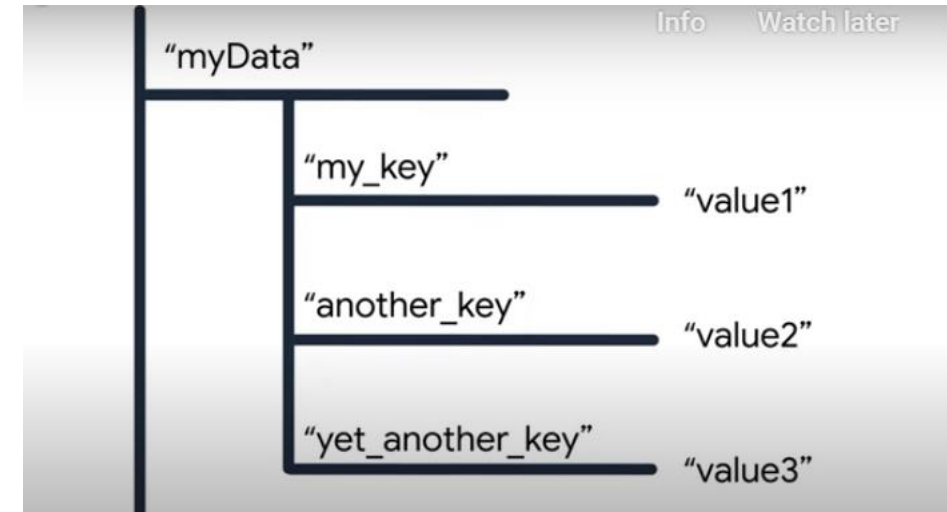
Monitor app usage and events with Firebase Analytics for better insights.

Visuals: Icons for each feature or a flowchart showing data sync and offline capabilities.

Data Structure in Firebase Realtime Database

NoSQL JSON Tree Model: Data is stored in a large JSON tree.

Simple to structure and flexible but requires careful organization to avoid duplication.



Firestore Realtime Database Challenges:

- Data duplication due to lack of complex querying.
- Flattened structure is often necessary for efficient reads
- Visuals: Sample JSON data structure showing nested objects and how data can be flattened.

Security and Firebase Authentication:

- Supports multiple sign-in methods (email, Google, Facebook, etc.).
- Simplifies user authentication and management for developers.

Firebase Security Rules:

- Allows control over who can read/write data in the database.
- Example rule: ``auth != null`` to restrict access to authenticated users.

Scaling with Firebase Realtime Database

Horizontal Scalability:

Firebase supports sharding, allowing apps to scale horizontally.

Load Balancing:

Firebase manages traffic across multiple databases automatically.

Offline Support in Firebase Realtime Database

How Offline Access Works:

Data is stored locally on the client device, enabling app use even without an internet connection.

Firebase syncs changes once the device is back online.

Benefits of Offline Support:

Ensures a seamless user experience in areas with poor connectivity.

Limitation in Firebase Realtime Database

Relational data models are not as well-suited, and complex queries are challenging.

Optimal for simpler, high-speed applications but requires careful structuring at scale.

Pros and Cons

Pros:

Easy setup, real-time updates, cross-platform support, and flexible security.

Cons:

Limited query complexity, potential for data duplication, security management complexity.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

Integration to Backend

Http Requests, Sending POST Requests

Dr. Annu Sharma
Department of Computer Applications

Http Requests, Sending POST Requests

In Flutter, you can send HTTP requests using the http package.

This package allows you to perform GET, POST, PUT, DELETE, and other HTTP requests seamlessly.

Http Requests, Sending POST Requests

HTTP (Hypertext Transfer Protocol) allows communication between a client and a server.

Types of requests:

- **GET: Fetch data.**
- **POST: Send data.**
- **PUT/PATCH: Update data.**
- **DELETE: Remove data.**

API = Application Programming Interface

- An API allows two software applications to **talk to each other**
- It works like a **messenger** that carries a request from your app → server
- Then brings a **response** back from the server → your app
- Flutter uses APIs to communicate with online databases, services, or backend systems

Example:

Flutter App → asks server: “Give me list of users”

Server → sends back user data

Why Do We Use APIs?

- To get data from a server (like weather, students, products)
- To send data to a server (login, registration, upload)
- To update existing information
- To delete records
- To connect mobile apps to databases

APIs make apps **dynamic**, **connected**, and **powerful**.

Types of API Requests

◆ GET Request

- Used to **fetch data**
- Example:
 - Get list of students
 - Get weather today
 - Get products

◆ POST Request

- Used to **send data** to server
- Example:
 - Register a user
 - Submit a form
 - Add new record to database

Mobile App Development

Integration to Backend



JSON = JavaScript Object Notation

It is the format in which APIs send and receive data.

- Lightweight
- Easy to read
- Looks like a dictionary or map
- Flutter converts it using `jsonDecode()` and `jsonEncode()`

Example JSON from an API

<https://jsonplaceholder.typicode.com/users>

Why JSON is Used in APIs

- Easy for server to generate
- Easy for Flutter to convert into objects
- Lightweight → fast
- Universal format → works with all programming languages

API Flow Diagram

Flutter App -----> Server
GET

Flutter App <----- Server
JSON Response

Flutter App -----> Server
POST (data)

Flutter App <----- Server
Success Response

Http Requests, Sending POST Requests

- Use `http.get()` for fetching
- Use `http.post()` for uploading
- Convert JSON using Dart's `jsonDecode()`
- Display data using `ListView`
- Test APIs using `jsonplaceholder` for practice

Http Requests, Sending POST Requests

http Package

- A Flutter/Dart library used to communicate with web servers
- Helps you send requests and receive data from APIs
- Supports:
 - **GET** (fetch data)
 - **POST** (send data)
 - **PUT** (update)
 - **DELETE** (remove)

To use it:

dependencies: http: ^1.2.0

Import:

```
import 'package:http/http.dart' as http;
```

Http Requests, Sending POST Requests

http.get() and (Fetch Data)

GET = Retrieve data from server.

Example:

```
final response = await http.get( Uri.parse("https://jsonplaceholder.typicode.com/users") );
```

How it works:

- 1.Flutter sends GET request
- 2.Server receives it
- 3.Server retrieves data
- 4.Server sends data in **JSON format**
- 5.Flutter app displays it

GET Request Example

- Use `http.get` to fetch data from an API.

```
Future<void> fetchData() async {  
    final response = await  
    http.get(Uri.parse('https://api.example.com/data'));  
    if (response.statusCode == 200) {  
        print(response.body);  
    } else {  
        print('Error: ${response.statusCode}');  
    }  
}
```

Http Requests, Sending POST Requests

http.post() And (Send Data)

POST = Send data to the server.

Example:

```
final response = await http.post(  
  Uri.parse("https://jsonplaceholder.typicode.com/posts"), headers: {"Content-  
Type": "application/json"}, body: jsonEncode({ "title": "Hello", "body": "This is my  
first POST" }), );
```

Sending a POST Request

Use `http.post` to send data to the server.

```
Future<void> sendData() async {  
    final response = await http.post(  
        Uri.parse('https://api.example.com/create'),  
        headers: {'Content-Type': 'application/json'},  
        body: jsonEncode({'key': 'value'}),  
    );  
    if (response.statusCode == 201) {  
        print('Success: ${response.body}');  
    } else {  
        print('Error: ${response.statusCode}');  
    }  
}
```

GET Request Works

- 1.Flutter app sends GET request
“Give me all users”
- 2.Server receives request
- 3.Server fetches data from its database
- 4.Server sends data back in **JSON format**
- 5.Flutter displays the results

How POST Request Works

- 1.Flutter app sends POST request with data
name = “Rahul”, email = “rahul@gmail.com”
- 2.Server saves data in database
- 3.Server returns **success message** or newly created record
- 4.Flutter displays confirmation

Handling JSON Responses

"Working with JSON"

- Use `dart:convert` for JSON encoding and decoding.

```
import 'dart:convert';
```

```
void processResponse(String responseBody) {  
  final data = jsonDecode(responseBody);  
  print('Name: ${data['name']}');  
}
```

Http Requests, Sending POST Requests

Flutter uses the http package to handle these requests.

"Adding the `http` Package"

1. Add the `http` package to `pubspec.yaml`:

dependencies:

flutter:

 sdk: flutter

 http: ^1.2.0

2. Write the code in your main.dart:

3. Import the package in your Dart file:

```
import 'package:http/http.dart' as http;
```

Mobile App Development

Integration to Backend



Program for Http Requests, Sending POST Requests

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ApiDemo(),
    );
  }
}
```

Mobile App Development

Integration to Backend



Program for Http Requests, Sending POST Requests

```
class ApiDemo extends StatefulWidget {  
  @override  
  State<ApiDemo> createState() => _ApiDemoState();  
}
```

```
class _ApiDemoState extends State<ApiDemo> {  
  List users = [];  
  String postResult = "";
```

```
  TextEditingController nameController =  
  TextEditingController();  
  TextEditingController emailController =  
  TextEditingController();
```

```
  final String baseUrl =  
  "https://jsonplaceholder.typicode.com";
```

Compiled By : Annu Sharma - Department of Computer Applications

```
//----- HTTP GET -----
```


Program for Http Requests, Sending POST Requests

```
//----- HTTP GET -----  
Future<void> fetchUsers() async {  
    final response = await  
http.get(Uri.parse("$baseUrl/users"));  
  
    if (response.statusCode == 200) {  
        setState(() {  
            users = jsonDecode(response.body);  
        });  
    } else {  
        throw Exception("Failed to load users");  
    }  
}  
  
//----- HTTP POST -----
```

Mobile App Development

Integration to Backend



Program for Http Requests, Sending POST Requests

```
//----- HTTP POST -----  
Future<void> createUser() async {  
    final response = await http.post(  
        Uri.parse("$baseUrl/users"),  
        headers: {"Content-Type": "application/json"},  
        body: jsonEncode({  
            "name": nameController.text,  
            "email": emailController.text,  
        }),  
    );  
    if (response.statusCode == 201) {  
        setState(() {  
            postResult = response.body;  
        });  
    } else {  
        throw Exception("Failed to create user");  
    }  
}
```

```
//-----
```

Mobile App Development

Integration to Backend



Program for Http Requests, Sending POST Requests

```
//-----  
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text("HTTP GET & POST  
Example")),  
    body: Padding(  
      padding: const EdgeInsets.all(16),  
      child: Column(  
        children: [  
          // GET USERS BUTTON  
          ElevatedButton(  
            onPressed: fetchUsers,  
            child: Text("GET USERS"),  
          ),  
          // SHOW USERS LIST
```

Program for Http Requests, Sending POST Requests

```
// SHOW USERS LIST
Expanded(
  child: ListView.builder(
    itemCount: users.length,
    itemBuilder: (context, index) {
      return ListTile(
        title: Text(users[index]["name"]),
        subtitle: Text(users[index]["email"]),
      );
    }, ), ),
// POST FIELDS
TextField(
  controller: nameController,
  decoration: InputDecoration(
    labelText: "Enter Name",
    border: OutlineInputBorder(),
  ), ),
```

Program for Http Requests, Sending POST Requests

```
        SizedBox(height: 10),

        TextField(
          controller: emailController,
          decoration: InputDecoration(
            labelText: "Enter Email",
            border: OutlineInputBorder(),
          ),
        ),
        SizedBox(height: 10),

        // POST BUTTON
        ElevatedButton(
          onPressed: createUser,
          child: Text("POST - Create User"),
        ),
        SizedBox(height: 10),
```

```
        SizedBox(height: 10),

        // SHOW POST RESULT
        Text("POST Response: $postResult"),
      ],
    ),
  ),
);
}
}
```

Mobile App Development

Integration to Backend

Program for Http Requests, Sending POST Requests

OUTPUT



Http Requests, Sending POST Requests

1. What does an HTTP GET request do?

- A. Inserts data
- B. Deletes data
- C. Fetches data from server
- D. Updates data

2. Which function is used to decode JSON in Dart?

- A. jsonConvert()
- B. jsonDecode()
- C. decodeJSON()
- D. convertJSON()

Http Requests, Sending POST Requests

3. What does status code 200 indicate?

- A. Not Found
- B. Bad Request
- C. Success
- D. Unauthorized

4. What is required to convert Dart Map into JSON string?

- A. `jsonEncode()`
- B. `mapToJson()`
- C. `convertJSON()`
- D. `writeJSON()`

Http Requests, Sending POST Requests

1. What does an HTTP GET request do?

Answer: C Fetches data from server

2. Which function is used to decode JSON in Dart?

Answer: B jsonDecode()

3. What does status code 200 indicate?

Answer: C Success

4. What is required to convert Dart Map into JSON string?

Answer: A jsonEncode()



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science